Отжиг и эвристические алгоритмы

Как работать с заданиями

Дано 3 оптимизационные задачи. В каждой задаче нужно вывести допустимое решение и постараться сделать его как можно лучше. Точное оптимальное решение находить и доказывать его оптимальность не требуется.

Некоторые задачи созданы под методы и алгоритмы, которые не обсуждались на парах. В условиях написано, с чего можно начать изучение и какие материалы помогут разобраться в теме.

Открытые тесты, примеры и скрипт локальной проверки `evaluate.py` лежат в папке `practice` репозитория работы; как ими пользоваться, описано в `practice/README.md`.

В качестве результата вы предоставляете отчет о своих решениях и код в Яндекс.Контест. Отчет должен быть информативным; ответы на вопросы должны быть по существу.

В заданиях упомянуты некоторые статьи, на которые вы можете опираться при выполнении работы.

Использование ИИ

Эта работа AI-friendly. Разрешено использовать нейросети для объяснения терминов и формул, поиска литературы, разработки алгоритмов, исправления кода, подготовки тестов и оформления отчёта.

Проверяйте материалы, на которые ссылается ИИ: проверьте существование источника и прочитайте нужный фрагмент. Полезно попросить ИИ объяснить его на маленьком примере, а затем самостоятельно проверить этот пример. Если объяснение или алгоритм вызывает сомнение, попробуйте найти контрпример либо сравните с точным перебором на малых данных.

Вы отвечаете за всё, что написано в отчёте: утверждения должны соответствовать вашему коду и вашим запускам.

Примерный пайплайн работы

Для каждой задачи

1. Получите простое допустимое решение, сохраните его как базовую версию и **отправьте её в Контест**.
2. Изучите понятия и методы из раздела «Что изучить» в условии задачи. Выберите и реализуйте хотя бы одно содержательное изменение алгоритма. Само изучение метода не означает, что именно его нужно оставить в итоговой программе.
3. Сравните две версии на двух-трёх открытых тестах с разными входными данными. Если метод случайный, выполните по три запуска с заранее выбранными seed. Запишите качество, время и краткий вывод; для случайного метода укажите медиану и диапазон результатов.
4. Сохраните ссылки на использованные материалы и отметьте, какую идею взяли из каждого и в каком месте программы она реализована.

Неудачная попытка тоже полезна, если вы можете показать результат и объяснить возможную причину. Улучшение не требуется приписывать методу, если эксперимент его не подтвердил.

Что такое отжиг в этих заданиях

Состояние — ваш текущий набор предметов или маршрут. Локальный ход немного меняет состояние: например, заменяет предмет или переносит точку маршрута. Энергия E — число, которое алгоритм старается уменьшить.

Если $\Delta = E_{\text{new}} - E_{\text{current}} \le 0$, новый вариант принимается. Если $\Delta > 0$, стандартный вариант отжига принимает его с вероятностью

$$P = \exp(-\Delta/T),$$

где $T > 0$ — температура. Постепенное уменьшение температуры снижает вероятность принятия ухудшений. Это правило не гарантирует нахождение оптимума за ограниченное время.

Температура должна соответствовать масштабу изменений энергии. Число «1000» само по себе ничего не означает: исследуйте типичные для Δ значения. Храните лучшее допустимое решение отдельно от текущего и выводите именно его. Если используете штрафы и временно разрешаете недопустимые состояния, финальный ответ всё равно обязан выполнять все ограничения.

Баллы за задачи

Каждая задача оценивается максимум в 100 баллов. Предусмотрены 10 оцениваемых тестов по 10 баллов. Для каждого теста до начала соревнования было зафиксировано положительное значение R (ориентир) — качество допустимого решения организаторов.

Пусть Q — качество вашего допустимого ответа в единицах соответствующей задачи. На одном тесте начисляется

$$s = 10\rho^{10}.$$

В задаче A Q нужно максимизировать, поэтому $\rho = \min(1, Q/R)$. В задачах B и C Q нужно минимизировать, поэтому $\rho = \min(1, R/Q)$.

Ответ лучше ориентира получает 10 баллов. Нарушение формата или ограничений ответа даёт 0 на соответствующем тесте. Итог — сумма по 10 тестам. В зачёт идёт лучшая суммарная посылка по каждой задаче; результаты разных посылок по отдельным тестам не объединяются.

Вычисления с вещественными числами допускают погрешность округления. В чекере значение $\rho \ge 1 - 10^{-12}$ считается равным 1. Длина маршрута и сумма логарифмов вычисляются чекером, эти числа вам выводить не нужно.

Сдача работы

До **23:59 26 октября** заполните форму сдачи. Форма заполняется один раз; если нужно исправить ответ, отправьте её повторно — проверяется последняя отправка, сделанная до дедлайна. Посылки в Контест после дедлайна не учитываются.

В форме нужно:

1. Указать фамилию, имя, отчество, группу и логин в Контесте.
2. Приложить отчёт в формате PDF, файл `Фамилия_Имя.pdf`. Его устройство описано ниже.
3. Приложить лог общения с ИИ: файл `Фамилия_Имя_ai_log` с расширением `.pdf`, `.md`, `.txt` или `.json`. Подойдёт экспорт чата или сохранённая в PDF страница переписки. Если чатов

или инструментов было несколько, соберите логи в один архив `Фамилия_Имя_ai_log.zip`. Перед отправкой удалите из логов пароли, токены и личные данные, не относящиеся к работе.

Отчёт

В начале отчёта приведите таблицу посылок: для каждой задачи — номер (ID) **итоговой** посылки, то есть лучшей по сумме баллов, и номер посылки **базовой** версии.

Посылка	A	B	C
Итоговая (лучшая по сумме баллов)	1234890	…	…
Базовая	1234567	…	…

Далее для каждой из задач A, B, C отчёт содержит пять разделов.

1. **Модель.** Что именно оптимизируется, какие ответы допустимы и где в вашей программе гарантируется допустимость выведенного ответа.
2. **Алгоритм итоговой посылки.** Начальное решение, как вы пришли к улучшенному решению, правило принятия, пересчёт качества при ходе и стоимость оценки и применения хода. Описывайте код именно той посылки, которая указана как итоговая в таблице посылок. Ссылайтесь на него по имени функции или номерам строк, например: «обмен — функция `try_swap`, строки 120–145».
3. **Эксперимент.** Таблица сравнения базовой и улучшенной версий и вывод из неё в 2–4 предложениях (образец таблицы ниже).
4. **Источники.** Одна-две ссылки; для каждой — какую идею вы взяли и в каком месте кода она реализована.
5. **Ответы на вопросы.** Письменные ответы на вопросы из конца условия задачи, по 2–4 предложения. Где вопрос касается вашей реализации, отвечайте на примере своего кода и своих данных, а не в общем виде. Где вопрос касается изменённой постановки (например, нулевые множители в A, несимметричные расстояния в B, сумма времён вместо максимума в C), укажите, что и в каком месте вашей программы пришлось бы изменить.

Образец таблицы эксперимента:

Версия	Тест	Лимит, с	Seed	Q	Время, с	Наблюдение
жадный (1234567)	`A/02.in`	1	—	1056,50	0,02	сортировка по $\ln p_i/w_i$
отжиг (1234890)	`A/02.in`	1	1, 17, 103	1056,87 [1056,87; 1056,87]	0,20	медиана [мин; макс]; все seed дали одно значение

Числа в таблице должны быть получены вашими запусками на открытых тестах из папки `practice` с помощью `evaluate.py` или своим кодом, если хотите. Укажите, на каком компьютере и с каким лимитом времени вы запускали программы.

Как проверяется работа

Работа (посылки в Яндекс.Контесте, отчёт и лог общения с ИИ) проверяется ассистентами.

Отчёт по каждой задаче оценивается отдельно по четырём критериям из таблицы ниже. По каждому критерию ставится 0, 0,5 или 1 балл, поэтому за задачу $X \in \{A, B, C\}$ получается оценка D_X от 0 до 4. Если отчёта по задаче нет, $D_X = 0$. Оценка за отчёт — среднее по трём задачам:

$$D = \frac{D_A + D_B + D_C}{3}.$$

Критерий	0 баллов	0,5 балла	1 балл
Модель и допустимость	Не объяснено, что оптимизируется и какие ответы допустимы	Основная цель описана, но есть существенная ошибка или пропуск	Правильно описаны цель, ограничения и место программы, где обеспечивается допустимость
Алгоритм и код	Описание не связано с кодом посылки или ему противоречит	Ход описан, но есть ошибка в пересчёте качества или в оценке сложности	Описаны ход, пересчёт качества и разумная оценка сложности со ссылками на код посылки
Материалы и эксперимент	Нет источника или сравнения, либо их нельзя проверить	Источник или сравнение есть, но вывод слабо следует из данных	Идея из источника связана с кодом, сравнение проведено честно и правильно интерпретировано
Ответы на вопросы	Ответов нет, они в основном неверны или написаны общими фразами	Часть ответов верна, но есть ошибки или ответы не связаны со своим кодом и данными	Ответы на все вопросы задачи верны и по существу, где нужно — со ссылкой на свой код или данные

Оценку снижают общие фразы, которые подходят к любой программе; утверждения, противоречащие коду посылки; несуществующие или непрочитанные источники; числа, которые не воспроизводятся при повторном запуске и не объяснены случайностью или лимитом времени. Если отчёт по задаче описывает не ту версию, что указана в таблице посылок, критерий «Алгоритм и код» для этой задачи оценивается по фактической посылке.

Итоговая оценка

За Контест можно получить 6 баллов итоговой оценки, за отчёт — 4. Если суммы баллов за задачи равны S_A, S_B, S_C, итог равен

$$\frac{S_A + S_B + S_C}{50} + D.$$

Баллы Контеста и отчёта не подменяют друг друга: без отчёта $D = 0$, но баллы Контеста сохраняются.

Задача A. Мультипликативный рюкзак

Для эксперимента нужно приобрести несколько модулей. Модуль i стоит w_i единиц бюджета и увеличивает условный показатель установки в p_i раз. По принятой в задаче модели эффекты выбранных модулей перемножаются. Каждый модуль можно приобрести не более одного раза.

Бюджет равен W. Выберите набор S, для которого

$$\sum_{i \in S} w_i \le W,$$

а произведение $\prod_{i \in S} p_i$ как можно больше. Пустой набор допустим; его произведение равно 1.

Это задача на качество решения. Выведите найденный набор.

Входные данные

Первая строка содержит целые числа N и W — количество модулей и бюджет: $2 \le N \le 2000$, $1 \le W \le 10^{12}$.

В следующих N строках заданы целые числа w_i и p_i: $1 \le w_i \le \min(W, 10^9)$, $2 \le p_i \le 10^9$. Модули нумеруются от 1 до N в порядке ввода. Гарантируется, что сумма всех стоимостей больше W.

Выходные данные

Выведите количество выбранных модулей m, затем m различных номеров модулей от 1 до N в любом порядке. Для пустого набора достаточно вывести `0`. Суммарная стоимость выбранных модулей не должна превышать W. Других чисел и пояснений в ответе быть не должно.

Пример

Ввод:

```
5 10
6 40
5 30
4 20
3 12
2 5
```

Один из допустимых ответов:

```
3
2 4 5
```

Стоимость равна $5 + 3 + 2 = 10$, произведение — $30 \cdot 12 \cdot 5 = 1800$.

Как вычисляется качество

Чекер использует $Q(S) = \sum_{i \in S} \ln p_i$. Натуральный логарифм строго возрастает, поэтому максимизация суммы логарифмов эквивалентна максимизации произведения.

Пустой набор корректен, соответственно для него $Q = 0$.

Баллы нормируются именно по сумме логарифмов, согласно общим правилам. Произведение $\prod_{i \in S} p_i$ хранить и выводить не нужно.

Стоимости и их суммы храните в большом целочисленном типе. Обычная таблица динамики по всем значениям веса будет непрактичной на крупных тестах; другие точные и приближённые подходы разрешены.

Что изучить

Начальный подход: отсортировать модули по $\ln p_i/w_i$ и последовательно добавлять помещающиеся. Эта эвристика не гарантирует лучший набор. Проверьте это на примере из условия: такой порядок может дать набор с произведением 1200 вместо 1800.

Для улучшения изучите добавление, удаление и обмен «один выбранный на один невыбранный». Затем выберите хотя бы одно развитие: обмен «один на два», случайные перезапуски, возмущение с восстановлением допустимости или отжиг. Изучите правило принятия отжига, даже если по результатам сравнения выберете другой метод.

Для минимизации можно взять энергию $E = -Q$. Смена представления энергии сохраняет оптимальный набор, но не обязана сохранять поведение отжига: вероятность принятия зависит от численного значения разности энергий.

Что попробовать и сравнить

Сохраните жадную версию и сравните её с выбранным улучшением на данных с разными соотношениями стоимости и полезности. На малых наборах проверьте качество полным перебором. В собственных тестах или генераторе, который предложит ИИ, проверьте выполнение ограничений и отсутствие переполнения.

Поддерживайте текущую стоимость и Q так, чтобы оценивать одиночное добавление, удаление или обмен за $O(1)$. При штрафе за превышение бюджета объясните его смысл и отдельно сохраняйте лучший допустимый набор.

Для дополнительного эксперимента можно сравнить несколько начальных температур либо обмены «один на один» и «один на два». Меняйте один компонент за раз и предоставляйте версиям одинаковое время.

С чего начать чтение

Сначала разберите объяснения выше и попросите ИИ показать работу жадного выбора, обмена и отжига на пяти модулях из примера. Проверьте стоимость и произведение вручную.

Затем откройте работу U. Pferschy, J. Schauer, C. Thielen *The Product Knapsack Problem: Approximation and Complexity*. Для этой задачи достаточно прочитать постановку и найти обсуждение логарифмов и жадного подхода. В статье рассматриваются и отрицательные множители; в нашем задании все $p_i \geq 2$, поэтому эта часть не требуется.

Вопросы для отчёта

1. Почему можно заменить произведение суммой логарифмов? Что изменилось бы при нулевых или отрицательных множителях?
2. Как обновляется значение Q при обмене? Какова стоимость каждого хода в вашей реализации?
3. Если умножить энергию на 10^6, как нужно изменить температуру, чтобы сохранить те же вероятности принятия? Какие строки вашей программы пришлось бы изменить?

Задача B. Курьер и районы города

Курьер должен посетить все пункты выдачи и вернуться в исходный пункт. В некоторых частях города пункты расположены плотными группами, в других — вдоль улиц или далеко друг от друга. Такое устройство данных можно использовать при построении маршрута.

Даны N точек на плоскости. Курьер начинает в точке 1, посещает каждую из точек $2, \ldots, N$ ровно один раз и возвращается в точку 1. Между любыми двумя точками можно перемещаться напрямую. Длина перехода равна обычному евклидову расстоянию

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Найдите замкнутый маршрут как можно меньшей длины.

Входные данные

Первая строка содержит целое число N, такое что $3 \le N \le 5000$. В следующих N строках находятся целые координаты x_i, y_i ($|x_i|, |y_i| \le 10^6$). Все точки различны. Нумерация соответствует порядку ввода.

Во входе нет номеров районов, числа кластеров или параметров кластеризации. Среди тестов предусмотрены компактные группы, группы разной плотности, вытянутые группы, отдельные точки между группами и данные без очевидного кластерного разбиения. Находить группы и решать, как использовать их при построении маршрута (и использовать ли вообще), нужно самостоятельно.

Выходные данные

Выведите $N+1$ целое число — номера точек в порядке обхода. Первое и последнее числа должны быть равны 1. Каждый номер от 2 до N должен встречаться ровно один раз; номер 1 встречается только в начале и конце. Длину маршрута выводить не нужно.

Пример

Ввод:

```
6
0 0
0 2
2 2
2 0
10 0
10 2
```

Один из допустимых ответов:

```
1 2 3 6 5 4 1
```

Длина этого маршрута равна $2 + 2 + 8 + 2 + 8 + 2 = 24$.

Как вычисляется качество

Качество Q — сумма длин всех N переходов, включая возвращение в точку 1. Чем меньше Q, тем лучше. Баллы рассчитываются по общим правилам для минимизации.

Используйте обычные расстояния. Замена каждого расстояния его квадратом меняет задачу. При вычислении квадратов разностей координат избегайте переполнения целочисленного типа.

Что изучить

Евклидова задача коммивояжёра; ближайший сосед; локальный поиск 2-opt; кластеризация DBSCAN; соединение локальных маршрутов; изменение решения за границами кластеров.

Начните с маршрута по ближайшей непосещённой точке. Затем разберите 2-opt: замените рёбра (a, b) и (c, d) на (a, c) и (b, d), развернув соответствующий участок маршрута. Для симметричных расстояний изменение длины равно

$$\Delta = d(a, c) + d(b, d) - d(a, b) - d(c, d).$$

Все внутренние рёбра участка сохраняют длину при обращении. Проверка одного хода занимает $O(1)$, разворот участка обычного массива — $O(k)$ для участка длины k. Оцените также стоимость перебора всех пар и память для полной матрицы расстояний.

Изучите назначение `eps` и `min_samples` в DBSCAN, различие основных и граничных точек, понятие шума. Шум в кластеризации не означает, что точку можно пропустить в маршруте.

Кластеризацию нужно попробовать как способ построения или улучшения маршрута. Можно использовать DBSCAN, k-means, пространственную сетку или другой способ разбиения. Итоговую версию выбирайте по измеренному качеству.

Что попробовать и сравнить

Сохраните вариант «ближайший сосед + локальное улучшение». Постройте второй вариант, использующий разбиение точек на группы. Определите порядок групп, выберите точки входа и выхода и соедините пути внутри групп. После соединения разрешите улучшения всего маршрута, в том числе между группами.

Не считайте автоматически оптимальными обход центров групп или независимые замкнутые циклы внутри них. Проверьте, как меняются реальные переходы между точками при выборе входа и выхода.

Сравните версии на компактных группах и на данных с разной плотностью либо отдельными точками. Покажите в отчёте хотя бы один пример, где выбранное разбиение помогает или мешает; отсутствие улучшения допустимо, если оно подтверждено сравнением. Картинка маршрута на таком примере поясняет вывод лучше длинного текста :)

Для дальнейшего исследования выберите одну идею: отжиг с разворотами, перенос отдельной точки, k-NN или смена масштаба кластеризации.

С чего начать чтение

В руководстве scikit-learn по DBSCAN прочитайте объяснение основных точек, параметров и шума. Рассмотрите иллюстрации поведения кластеризации. Код из документации служит для изучения: наличие scikit-learn в среде Контеста не предполагается.

В обзоре D. S. Johnson, L. A. McGeoch *The Traveling Salesman Problem: A Case Study in Local Optimization* найдите раздел о 2-opt и раздел об отжиге или повторном локальном поиске.

Вопросы для отчёта

1. Почему формула для Δ использует только четыре расстояния? Что изменится для несимметричных переходов и что пришлось бы поменять в вашей программе?
2. Что делают параметры выбранной кластеризации и какие значения вы использовали? Как обрабатываются точки вне кластеров?

3. Как выбираются вход и выход из группы и как соединяются группы?

Задача C. Собрать заказы к концу смены

На складе нужно выполнить N заданий по сборке товаров. Для этого доступны K сотрудников. Все они начинают работу одновременно в пункте выдачи заданий с номером 0. После выполнения своих заданий каждый сотрудник должен вернуться в пункт 0.

Для каждой пары точек известно время перехода, а для каждого задания — время сборки. Требуется распределить задания между сотрудниками и определить порядок выполнения так, чтобы вся работа завершилась как можно раньше.

Каждое задание нужно назначить ровно одному сотруднику и выполнить ровно один раз. Разные сотрудники работают параллельно. Каждый может выполнить любое задание; препятствия, проходы и затраты времени на переходы уже учтены во входной матрице. Ожидания из-за других сотрудников, ограничения вместимости и новые заказы во время работы не моделируются.

Входные данные

Первая строка содержит целые числа N и K ($2 \le N \le 500,\ 1 \le K \le \min(20, N)$).

Вторая строка содержит N целых чисел $s_1, \ldots, s_N$ — время сборки i-го задания в секундах ($1 \le s_i \le 3600$).

Следующие $N + 1$ строк содержат по $N + 1$ целому числу t_{ij} — кратчайшее время перехода из точки i в точку j, в секундах. Строки и столбцы нумеруются от 0 до N. Гарантируется:

- $t_{ii} = 0$;
- $1 \le t_{ij} \le 10^6$ при $i \ne j$;
- $t_{ij} = t_{ji}$;
- $t_{ij} \le t_{ih} + t_{hj}$ для любых i, j, h.

Точка $i \ge 1$ соответствует заданию i. Время t_{ij} не включает сборку. Проход мимо другого задания не означает его выполнение. Матрица уже задана, поэтому восстанавливать план склада и искать кратчайшие пути для чтения входа не требуется.

Выходные данные

Выведите K строк. Строка k-го сотрудника начинается с числа m — количества заданий, которые он выполнит. Далее идут m чисел — номера этих заданий в порядке выполнения. Если сотруднику не досталось заданий, его строка состоит из одного числа `0`.

Каждый номер от 1 до N должен встретиться во всех строках вместе ровно один раз. Номер пункта 0 не выводится: начальный выход из него и возвращение подразумеваются. Времена работы выводить не нужно.

Пример

Ввод:

```
4 2
2 1 2 1
0 3 2 2 3
3 0 1 5 6
2 1 0 4 5
2 5 4 0 1
3 6 5 1 0
```

Один из допустимых ответов:

```
2 1 2
2 3 4
```

Первый сотрудник выполняет 2 задания: сначала задание 1, затем задание 2. Второй выполняет задания 3 и 4. Первый сотрудник затрачивает $3 + 2 + 1 + 1 + 2 = 9$ секунд: переход к заданию 1, сборка, переход к заданию 2, сборка и возвращение. Второй также затрачивает 9 секунд. Вся работа завершается через 9 секунд.

Как вычисляется качество

Пусть k-й сотрудник выполняет m заданий с номерами $v_1, \ldots, v_m$ в этом порядке. Время его работы

$$T_k = t_{0v_1} + \sum_{j=1}^{m-1} t_{v_j v_{j+1}} + t_{v_m 0} + \sum_{j=1}^{m} s_{v_j}.$$

Если у сотрудника нет заданий ($m = 0$), то $T_k = 0$. Качество всего ответа

$$Q = \max(T_1, \ldots, T_K).$$

Чем меньше Q, тем лучше. Баллы вычисляются по общим правилам для минимизации. Требуется уменьшать время последнего закончившего сотрудника. Равное количество заданий и минимальная сумма времён маршрутов сами по себе не решают эту задачу.

Что изучить

Задача маршрутизации нескольких исполнителей; максимальное время завершения, или makespan; балансировка нагрузки; вставка задания в маршрут; перенос и обмен заданий между маршрутами.

Начните с последовательного назначения заданий. Для очередного задания можно перебрать сотрудников и места вставки, оценить новые времена и выбрать вариант с наименьшим новым максимумом. Объясните, как разрешаются равенства. Сравните такой выбор с назначением сотруднику, который сейчас занят меньше всех.

Разберите увеличение времени при вставке задания v между соседями a, b:

$$\Delta T = t_{av} + t_{vb} - t_{ab} + s_v.$$

На концах маршрута используется пункт 0. Для пустого маршрута положите $a = b = 0$.

Изучите хотя бы один способ выхода из локального оптимума: отжиг, случайные возмущения или large neighborhood search (LNS). В простом варианте LNS удаляет из решения несколько заданий и заново вставляет их в подходящие места. Если новое допустимое решение хуже, его принятие зависит от выбранной вами стратегии.

Что попробовать и сравнить

Сохраните базовое распределение. В улучшенной версии используйте хотя бы один ход между сотрудниками: перенос задания, обмен двух заданий или обмен участками маршрутов. Можно дополнительно менять порядок заданий внутри каждого маршрута.

Сравните работу на данных, где основную нагрузку создают переходы, и на данных, где существенны времена сборки. Проверьте, помогает ли уменьшение самого длинного маршрута и не возникает ли другой перегруженный сотрудник.

Для дополнительного исследования сравните локальные изменения с удалением и повторной вставкой группы заданий. Подберите размер группы экспериментально. При фиксированном времени большая перестройка может давать более сильный ход, но оставляет меньше попыток.

На маленьких данных проверьте стоимость маршрутов вручную и сравните с перебором.

Литература

В описании Vehicle Routing Problem в OR-Tools прочитайте постановку и объяснение минимизации самого длинного маршрута. Обратите внимание, что в примере библиотеки используются составляющие целевой функции: для корректного сравнения проверяйте фактически оптимизируемую величину.

В Routing Options посмотрите раздел о стратегиях локального поиска и ограничении времени.

Запросы для дальнейшего поиска: `min max multiple traveling salesman`, `vehicle routing relocate swap`, `large neighborhood search destroy repair`. Выберите один понятный вам метод и сохраните источник, который помог применить его.

Вопросы для отчёта

1. Почему одинаковое количество заданий может давать сильно различающиеся времена работы? Приведите пример из своих запусков или маленький пример с числами.
2. Как вставка или перенос задания меняет время маршрута? Где в вашем коде учитывается сборка?
3. Как вы обновляете общий максимум и обрабатываете пустой маршрут?
4. Что изменится, если вместо максимума времён потребуется минимизировать их сумму? Какие части вашей программы останутся полезными, а какие станут лишними?